Description, implementation and documentation of an artificial signal generator in Python

Federico Menegazzo

October 21, 2026

1 Introduction

Note 1.1. *Given A and B two non-empty sets, will denote with B^A the set of all possibile functions $f : A \to B$*

Definition 1.1. *Given $\mathcal{N} : \mathbb{R}^{\mathbb{R}} \to \mathbb{R}^{\mathbb{R}}$, we will say that: 'N is a normalization function', if:*

- $\forall c \in \mathbb{R},\ \mathcal{N}(f + c) = \mathcal{N}(f)$
- $\forall \lambda \in \mathbb{R},\ \mathcal{N}(\lambda\, f) = \mathcal{N}(f)$

Definition 1.2. *Given $f \in C^2(\mathbb{R})$, we define $K_f : \mathbb{R} \to \mathbb{R}$, also called 'the curvature function of f', as follows:*

$$K_f = \frac{f''(x)}{[1 + f'(x)^2]^{\frac{3}{2}}}$$

Proposition 1.1. *Given $C : [a, b] \to \mathbb{R}$ the analytical expression for the arc of a circle of radius r, it is true that:*

$$\forall\, x \in [a, b],\ K_C(x) = \frac{1}{r}$$

Note 1.2. *It's worth noting that: any $x \in \mathbb{R}^n$, for some $n \in \mathbb{N}$, will be called either a 'signal' or a 'discrete signal' from here on out. For convenience's sake, we'll denote with $x[j]$ the j-th index of the signal x, where $j \in \{0, \ldots, n-1\}$*

Definition 1.3. *Let's define $Z : \mathbb{R}^n \to \mathbb{R}^n$, as follows:*

$$Z(v) = \frac{v - v[0]}{\|v - v[0]\|_\infty}$$

which is a valid normalization function

Definition 1.4. *We will say that a signal x is 'normalized', if $Z(x) = x$*

In conclusion, the main goal of our signal generator is to take as input some abstract parameters, that will be later defined, and return a discrete, normalized and random signal f, according to the following model:

$$f = \beta\sigma\, s + \beta\omega\, o + \beta\gamma\, c + n \tag{1}$$

where:

- s, o and c are also discrete, normalized and random signals, which are, respectiveley, called the: 'smooth', 'highly oscillatory' and 'cartoon' components of our signal f
- β_σ, β_ω and β_γ are called 'blending factors'. All of them belong to $\mathbb{R}^+$ and also $\beta_\sigma + \beta_\omega + \beta_\gamma = 1$
- n is a discrete and random signal whose components have all been indipendently drawn from a Generalized Gaussian (GG) distribution, with some appropriate set parameters.

To this avail, within the framework of the Python language, a *signal* class will be implemented. Here's a collection and explanation of its public methods and attributes:

1.1 Attributes

- self.__n: represents the dimension of the discrete signal; i.e: the number of total indices needed to properly define the signal;
- self.__image: it's a *numpy* array of shape $(self.__n,)$, representing the actual values of the signal. At construct time, it's initialized to be filled with zeros;

1.2 Operations

Let s and t be two Signal objects and λ a Float number. Then:

- $s + t$: returns the component-wise sum of the two signals;
- $\lambda * s$: returns the components-wise scalar multiplication of the s signal;
- $s[j]$: returns the j-th component of the signal;
- $len(s)$: returns the length of the signal;

1.3 Methods

- self.get_image(): return a copy of the image of the signal;
- self.return_curvature_vect(): returns a numpy array filled with the values of the discrete curvatures at each index of the signal; except, for the first and last one, of course. See Section 2;
- self.generate_GG_realization(μ, σ, β): generates a signal whose components are i.i.d random values drawn from a Generalized Gaussian distributuion of parameters μ, σ and β. Better explained in Section 5;
- self.generate_smooth_sign(K, must_be_normalized=True): explained in Subsection 2.1;
- self.generate_oscillatory_sign(interpolant, n_ext, T_min, K, std_sin=False): explained in Subsection 3.1;
- self.generate_cartoon_sign(k, T_min, p=0.95): explained in Subsection 4.1;
- self.generate_blended_signal(snr_tuple, beta_tuple, T_min_cartoon, prob_std_sin, kernel='cos'): explained in Section 5

2 Generating the smooth component

Lemma 2.1. *Let $f : [a, b] \to \mathbb{R}$, $f \in C^2(\mathbb{R})$ e $K \in \mathbb{R}^+$, such that:*

- $f(a) = f(b) = 0$
- $f(x) \neq 0 \ \ \forall\, x \in (a, b)$
- $0 \leq |K_f(x)| \leq K$

Let now $C : [a, c] \subseteq [a, b] \to \mathbb{R}$, be the only circumference arc such that:

- $C(a) = 0 = C(c)$
- $C'(a) = f'(a)$
- $0 \leq |K_C(x)| = K \ \ \forall\, x \in [a, b]$

Then: $c - a \leq b - a$ and $\forall\, x \in [a, c] \ \ f(x) \geq C(x)$

Proof. Without loss of generality, let's assume that: $f'(a) > 0$.
First, observe that:

$$\int \frac{f''(x)}{(1 + f'(x)^2)^{1.5}}\, dx = \frac{f'(x)}{\sqrt{(1 + f'(x)^2)}}$$

therefore:

$$\int_a^x \frac{f''(t)}{(1+f'(t)^2)^{1.5}}\,dt = \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} - \frac{f'(a)}{\sqrt{(1+f'(a)^2)}} \iff$$

$$\frac{f'(x)}{\sqrt{(1+f'(x)^2)}} = \int_a^x \frac{f''(t)}{(1+f'(t)^2)^{1.5}}\,dt + \frac{f'(a)}{\sqrt{(1+f'(a)^2)}}$$

calling r the length of C's ray. we get:

$$-K(x-a) + \frac{f'(a)}{\sqrt{(1+f'(a)^2)}} \leq \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} \leq K(x-a) + \frac{f'(a)}{\sqrt{(1+f'(a)^2)}} \iff$$

$$-K(x-a) + \frac{C'(a)}{\sqrt{(1+C'(a)^2)}} \leq \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} \leq K(x-a) + \frac{C'(a)}{\sqrt{(1+C'(a)^2)}} \iff$$

$$-\frac{1}{r}(x-a) + \frac{c-a}{2\,r} \leq \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} \leq \frac{1}{r}(x-a) + \frac{c-a}{2\,r} \iff$$

$$\frac{C'(x)}{\sqrt{(1+C'(x)^2)}} \leq \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} \leq \frac{1}{r}(x-a) + \frac{c-a}{2\,r} \tag{2}$$

Now, note that, $\forall\ (x,y) \in \mathbb{R}^2$:

$$\frac{x}{\sqrt{1+x^2}} \leq \frac{y}{\sqrt{1+y^2}} \iff x \leq y$$

and that, $\forall\ x \in [a, \frac{a+c}{2}]$:

$$\frac{C'(x)}{\sqrt{(1+C'(x)^2)}} = -\frac{1}{r}(x-a) + \frac{c-a}{2\,r} \geq 0$$

Hence, we can deduce that:

- $f'(x) \geq C'(x) \geq 0\ \forall\ x \in [a, \frac{a+c}{2}]$
- $f(\frac{a+c}{2}) > C(\frac{a+c}{2})$

Let's now suppose, $\exists\, y \in [\frac{a+c}{2}, c]$ such that:

$$f(y) < C(y)$$

we can now infer the existence if $z \in (\frac{a+c}{2}, y)$, such that:

- $f(z) = C(z)$
- $f'(z) < C'(z) < 0$

But this is absurd, beacause it would mean that:

$$f'(z) < C'(z) \iff \frac{f'(x)}{\sqrt{(1+f'(x)^2)}} < \frac{C'(x)}{\sqrt{(1+C'(x)^2)}}$$

and this would break Equation 2.

Therefore, $f(x) \geq C(x)\ \forall\ x \in [a, c]$.

□

Lemma 2.2. *Let $f : [a, b] \to \mathbb{R}$, $f \in C^2(\mathbb{R})$ e $K \in \mathbb{R}^+$, such that:*

- $f(a) = f(b) = 0$
- $f(x) \neq 0\ \ \forall\ x \in (a, b)$
- $0 \leq |K_f(x)| \leq K$

Let now $C : [a, b] \to \mathbb{R}$, be the only circumference arc such that:

- $C(a) = 0 = C(b)$
- $0 \leq |K_C(x)| = K\ \ \forall\ x \in [a, b]$

then, $\forall\ x \in [a, b]\ \ f(x) \leq C(x)$

Proof. Without loss of generality, let's assume that: $f'(a) > 0$.

First, note that if $f'(a) > C'(a) > 0$, then Lemma 2.1 implies a logical contradiction.

In fact, if C_{tang} is the circumference arc tangent to f in a (like that described in Lemma 2.1) and a and c are the only roots of the C_{tang} function; then, by Lemma 2.1:

$$\forall x \in [a, c]\ \ f(x) \geq C_{tang}(x)$$

but it's clear that, in this case, $c > b$. Therefore, it would be impossibile for $f(b) = 0$ and that's a contradiction.

Now that we have proved: $0 < f'(a) < C'(a)$.

As in the previous proof, note that:

$$\frac{C'(x)}{\sqrt{1+C'(x)^2}} = -\frac{1}{r}(x-a) + \frac{b-a}{2r}$$

also notice:

$$\frac{C'(a)}{\sqrt{1+C'(a)^2}} > \frac{f'(a)}{\sqrt{1+f'(a)^2}}$$

Suppose it exists $\overline{x} \in (a,b)$ such that $f(\overline{x}) = C(\overline{x})$ e $f'(\overline{x}) > C'(\overline{x})$. Then:

$$\begin{aligned} &\exists\, z_1 \in (a,\overline{x}) \text{ tale che } f'(z_1) = C'(z_1) \Rightarrow \\ &\frac{C'(z_1)}{\sqrt{1+C'(z_1)^2}} = \frac{f'(z_1)}{\sqrt{1+f'(z_1)^2}} \end{aligned}$$

Observe that it always exists a point $y > \overline{x}$ such that $f(y) = C(y)$ and $f(z) > C(z)\ \forall\, z \in (\overline{x}, y)$. Note that:

$$\begin{aligned} &f'(y) < C'(y) \Rightarrow \\ &\exists\, z_2 \in (\overline{x}, y) \text{ tale che } f'(z_2) = C'(z_2) \Rightarrow \\ &\frac{C'(z_2)}{\sqrt{1+C'(z_2)^2}} = \frac{f'(z_2)}{\sqrt{1+f'(z_2)^2}} \end{aligned}$$

But the simultaneous existence of z_1 and z_2 is absurd because then:

$\exists z_3$ tale che:

$$\frac{1}{z_2 - z_1}\left[\frac{C'(z_2)}{\sqrt{1+C'(z_2)^2}} - \frac{C'(z_1)}{\sqrt{1+C'(z_1)^2}}\right] = \left(\frac{C'}{\sqrt{1+(C')^2}}\right)'(z_3) = K_C(z_3) = -\frac{1}{r}$$

which is in stark contradiction with one of the hypothesis.

Therefore, it cannot exist $\overline{x}$ and because, in a neighbourhood of a, $f \leq C$; then: $\forall\, x \in [a,b]\ f(x) \leq C(x)$. □

Definition 2.1. *Starting from Lemma 2.1 and Lemma 2.2, we can present the following definition of 'smooth' function.*

A function $f : [a,b] \to \mathbb{R}$ is said to be 'smooth' if:

$$\|K_f\|_\infty \leq \frac{2}{b-a}$$

Definition 2.2. *Given s a discrete signal in $\mathbb{R}^n$, we define the 'discrete curvature of s at index j' as follows:*

$$K_s[j] = 2\ Tan\left(\frac{\theta}{2}\right)$$

where, calling $P_- = (j-1, s[j-1])$, $P = (j, s[j])$ and $P_+ = (j+1, s[j+1])$:

- $\theta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$
- $\theta < 0 \iff s[j+1] < 2\,s[j] - s[j-1]$
- $\theta > 0 \iff s[j+1] > 2\,s[j] - s[j-1]$
- $|\theta|$ = *the angle between the* $\overline{PP_+}$ *segment and the line passing through the points* P_- *and* P

Note 2.1. *here's a brief comment on why Definition 2.2 should be proper discretization of the 'curvature function' introduced beforehand.*

Say, we have two segments $\overline{AP}$ and $\overline{PB}$, with identical length h and P as their only extremum in common. Also, as described in Definition 2.2, let's also assume some value for their associated θ angle.

It's quite easy to derive that the radius r_{tang}, of the circle tangent to those segments in the points A and B, should be equal to:

$$r_{tang} = h\ \frac{1}{|\,Tan\left(\frac{\theta}{2}\right)\,|}$$

Also, if we were to compute the radius r_{inter} of the circle interpolating the points A, P and B, then:

$$\lim_{h \to 0^+} r_{inter} = 2\ r_{tang}$$

assuming θ to stay constant as h varies.

Therefore, given an angle α such that:

$$2\ Tan\left(\frac{\alpha}{2}\right) = \frac{1}{R}\ h$$

if we were to construct a chain of segments, whose θ angles are constantly equal to α and whose lengths are constantly equal to h, then, as $h \to 0^+$ we would get better and better approximations of an arc of a circle of radius R.

An analogous argument would be true if, instead of fixing the segments' legths, we were to fix the spacing on the x-axis, between the extrema of such segments, to be equal to some h.

Note 2.2. *Given s a discrete signal in $\mathbb{R}^n$, we say it is 'smooth' if:*

$$\|K_s\|_\infty \leq \frac{2}{n}$$

Note 2.3. *Given s and t two smooth signals in $\mathbb{R}^n$, then for any $p \in [0,1]$:*

$$s\,p + t\,(1-p)$$

is still a smooth signal

2.1 self.generate_smooth_sign(K, must_be_normalized=True)

Let n be the dimension of the signal we aim to generate, then:

- K: should be a positive, real value;
- must_be_normalized: should be a Python Boolean;

it's first worth mentioning that K should be such that:

$$\frac{1}{(n/4)^2+1} \leq K \leq \frac{2}{n}$$

Of course, in the implementation, there are checks in place so that this first condition is always verified.

If must_be_normalized is set to True, here's a brief summary of how this signal's generation actually works:

- First, we pick an appropriate (but still random) starting index i_0;
- Secondly, at i_0, we set the signal to be equal either to 1 or -1, again, randomly;
- Thirdly, at i_0-1 and at i_0+1, we assign to our signal two values: $s[i_0-1]$ and $s[i_0+1]$, such that: $K_s[i_0] \leq K$ (See Definition 2.2)

Before explaining any further, please notice that: if s is a signal, $s[j-1]=x$ and $s[j]=y$; for some index j and some appropriate $(x,y) \in [-1,1]^2$. Then, I would say, its quite intuitive to see: $\exists\, z \in [-1,1]$, such that:

$$s[j+1]=z \text{ and } K_s[j] \leq K$$

Therefore, given those three starting values, we can 'propagate' our signal (first on all indeces on the left and then on the right of i_0), in such a way that guarantees that each triplet of consecutive indeces always has discrete curvature $\leq K$.

Finally, we need to guarantee that:

- our signal's values, will always be in $[-1,1]$
- $s[0]=0$

The first condition can be verified by choosing our signal's values according to a simple algorithm:

- suppose that $s[j] = x_0$;
- if $K_s[j-1] > K$, pick another x_0 and repeat;
- else, suppose also that $|K_s[i]| = K$, $\forall\ i \geq j$
- under these assumptions, compute $M = max_{i \geq j}|s[i]|$
- if $M \leq 1$, then x_0 is a valid assignment for $s[j]$
- else, pick another x_0 and repeat

Once we have generated our prototipe signal s, the second condition, can be verified by employing a second smooth signal t, such that:

$$s[0]\ p + t[0]\ (1-p) = 0, \text{ for some } p \in [0,1]$$

and that:

$$\|t\|_\infty = 1 \text{ with } t[i_0] = s[i_0]$$

Finally, applying Note 2.3 we get that:

$$z = s\ p + t\ (1-p)$$

is a signal with the following properties:

- $\|K_z\|_\infty < K$
- $\|z\|_\infty = 1$
- $z[0] = 0$

Instead, if must_be_normalized is set to False, we repeat the same procedure but we avoid imposing $s[0] = 0$, therefore, we skip the final 'averaging' step.

3 Generating the highly oscillatory component

Definition 3.1. *We define an 'interpolating function' as any $f : [0,1] \to [0,1]$ a function such that:*

- $f(0) = 0$
- $f(1) = 1$
- *f is monotonely increasing*

Definition 3.2. *Let f be an interpolating function. Given (P_x, P_y) and (Q_x, Q_y) points in $\mathbb{R}^2$, such that $P_x \leq Q_x$ we say that the function:*

$$I_f : [P_x, Q_x] \to [P_y, Q_y]$$

$$I_f(x) = Q_y \; f\left(\frac{x - P_x}{Q_x - P_x}\right) + P_y \; \left[1 - f\left(\frac{x - P_x}{Q_x - P_x}\right)\right]$$

'f-interpolates' the points P and Q

3.1 self.generate_oscillatory_sign(interpolating_function, n_ext, K, T_min=10, std_sin=False)

Let n be the dimension of the signal we aim to generate. Then:

- interpolating_function: should be a Python lambda function;
- n_ext: should be a natural number;
- K: should be a positive, real value;
- T_min: should be a natural number;
- std_sin: should be a boolean value;

Here's a quick sketch of how the implementation works.

First, if std_sin is False, then the object:

```
smooth_modulus = signal(n)
smooth_modulus = self.generate_smooth_signal(K,
                      must_be_normalized=False)
```

is created. Then, n_ext indeces are randomly picked, making sure that no two of them are less than T_min apart.

Then, we take each pair of successive indeces and we f-interpolate between the point (first index, z) $\in \mathbb{R}^2$ and the point (other index, $-z$) $\in \mathbb{R}^2$; where z is taken, in an alternating fashion to be equal to: $+1$ for the first pair of successive indeces, -1 for the second pair of successive indeces, $+1$ for the third pair of successive indeces and so on.

Finally, this signal made of successive f-interpolations is multiplied, component-wise, with the smooth_modulus signal.

Instead, if std_sin is True, then this last step will not occurr.

In either case, there's a final normalization step involving the Z function (see Definition 1.4)

4 Generating the cartoon component

Lemma 4.1. *Suppose you have a random variable X such that, $Im(X) \in \{1, 2, \dots, k\}$, where, for any $a \in \{1, \dots, k\}$:*

$$\mathbb{P}(X = a) = \frac{1}{k}$$

Consider now the, indipendent and identically distributed random variables $Y_k \sim X$, $k \in \{1, \dots, n\}$, for some $n \in \mathbb{N}$. Call $Y = (Y_1, \dots, Y_n)$ the associated random vector.

Define $X_b = \#\{j \in \{1, \dots, n\} \mid Y[j] = b\}$, for any $b \in \{1, \dots, k\}$.

Then:

$$\mathbb{P}(X_1 \geq 1, X_2 \geq 1, \dots, X_k \geq 1) = (-1)^{(k\%2)} \sum_{j=1}^{k} (-1)^j \binom{k}{j} \left(\frac{j}{k}\right)^n$$

Also, for any $p \geq 0.95$:

$$\mid min\{n \in \mathbb{N} \mid 1 - k \left(\frac{k-1}{k}\right)^n \geq p\} - min\{n \in \mathbb{N} \mid \mathbb{P}(X_1 \geq 1, X_2 \geq 1, \dots, X_k \geq 1) \geq p\} \mid \leq 1$$

4.1 self.generate_cartoon_sign(k, T_min, p=0.95)

Let n be the dimension of the signal we aim to generate. Then:

- k should be a natural number;
- T_min should be a natural number;
- p should be a real number in $[0, 1]$;

This method returns a piece-wise constant signal where the spacing between jumps is at least T_min indeces wide. Also, the value at each jump is determined by calling the $numpy.random.uniform(-1, 1)$ function.

k should be seen as representing the amount of uniformly distributed bins partitioning the interval $[-1, 1]$. Then we can define n, the number of jumps needed in our signal, as follows:

$n = min\{j \in \mathbb{N} \mid$ if j jumps were to occur in our signal, then
the probability of having at least one value per each of the bins is greater than $p\}$

In conclusion, a final normalization step is computed.

5 self.generate_blended_signal(snr_tuple, beta_tuple, T_min_cartoon, prob_std_sin, kernel='cos')

Let n be the dimension of the signal we aim to generate. Then:

- snr_tuple: should be an ordered Python tuple of positive real values;
- beta_tuple: should be an ordered Python tuple of positive real values;
- T_min_cartoon: should be a positive integer;
- prob_std_sin: should be a real number in $[0, 1]$;
- kernel: should be a Python string;

This method returns a signal as the one described in Equation 1.

First we define the following variables:

```
snr = numpy.random.uniform(snr_tuple[0], snr_tuple[1])
beta = numpy.random.uniform(beta_tuple[0], beta_tuple[1])

K_smooth = numpy.random.uniform( 1/( (n/4)**2 + 1 ), 2/n )
K_osc = numpy.random.uniform( 1/( (n/4)**2 + 1 ), 2/n )

n_ext = numpy.random.randint(int(n/20), int(n/10) )
n_partitions = numpy.random.uniform(1, 10)

std_sin = True if (np.random.uniform() <= prob_std_sin) else False
```

and the *interp* Python lambda function:

```
if (kernel == 'cos'):

 interp = lambda x: (-np.cos(x * np.pi) + 1)/2

elif (kernel == 'lin'):

 interp = lambda x: x

else:

 interp = lambda x: np.heaviside( (x - 1), 1 )
```

then, we generate three random and normalized signals for each one of the components:

```
s = signal(self.__n)
o = signal(self.__n)
c = signal(self.__n)


s.generate_smooth_sign(K=K_smooth)

o.generate_oscillatory_sign(interpolating_function=interp,
                            n_ext=n_ext, K=K_osc, std_sin=std_sin)

c.generate_cartoon_sign(n_partitions=n_partitions, T_min=T_min_cartoon)
```

and a random, uniformly distributed, set of three blending factors:

```
beta_s = np.random.uniform(0, 1)
beta_o = np.random.uniform(0, 1 - beta_s)
beta_c = 1 - beta_s - beta_o
```

let's define:

$$S = \beta\sigma\, s + \beta\omega\, o + \beta\gamma\, c$$

Finally, we need to generate the noise component of the signal.

The Generalized Gaussian distribution takes three parameters as input:

- a μ parameter for its expected value;
- a σ parameter for its standard deviation;
- a β parameter for its 'shape';

We will always set $\mu = 0$ and the value for the β parameter as already been defined.
σ, on the other hand, is based on the snr value:

$$\sigma = 10^{-\frac{\text{snr}}{20}}\ \frac{\|S\|_2}{\sqrt{n}}$$

let's call N the generated random signal of length n, whose components have been drawn from this particular GG distribution.

Therefore, this method will actually construct the following signal (given Z as in Definition 1.4):

$$f = Z(S + N)$$

and will also return all of its components as a Python tuple:

$$(s, o, c, N)$$